RESEARCH PROPOSAL

INVARIANT-SC

Identifiable Flow-Program Induction from Heterogeneous Supply-Chain Data

Erase the headers. Recover the physical process.

CORE PROPOSITION

Different enterprises use different ERP schemas, names, units, signs, and posting conventions. The physical system underneath is less arbitrary: material, inventory, production, waste, and transport events must satisfy shared flow laws. INVARIANT-SC proposes to recover an executable canonical supply-chain program from those laws - and to state formally when the semantics cannot be uniquely recovered.

Domain: AI for sustainable and resilient supply chains
Primary research type: structured representation learning, program induction, and identifiability
Intended use: faculty review, research-team alignment, and paper planning

Executive Summary

Supply-chain AI is often evaluated after data from multiple firms has already been cleaned, joined, and assigned consistent meanings. In practice, that integration step is a major bottleneck. A stock movement may appear as a positive quantity in one system and a negative posting in another; units may differ; receipts may be delayed; identifiers may not join cleanly; and the same physical event may be split across several tables. Standards such as GS1 EPCIS provide interoperable target models, but they do not automatically infer how arbitrary legacy tables map into those models [9, 10].

INVARIANT-SC reframes this integration problem as **latent flow-program induction**. The model is given heterogeneous relational tables and must infer semantic column roles, units, sign conventions, keys, joins, temporal offsets, and the executable stock-flow relations that connect them. The central source of supervision is not the header text; it is the set of conservation and accounting laws that the underlying physical process should satisfy.

RESEARCH CLAIM

A hybrid learned-and-symbolic system can recover supply-chain semantics that generalize across unseen schemas and can distinguish uniquely identifiable interpretations from observationally equivalent ones.

The proposed paper has five tightly connected contributions:

1. A new machine-learning task: **physics-grounded semantic schema induction** from anonymized, heterogeneous relational data.
2. A hybrid method combining learned structural proposals, differentiable invariant checking, robust temporal alignment, and an exact verifier.
3. Identifiability and impossibility results that characterize when roles, units, joins, and lags are recoverable.
4. An open stress-test benchmark built from supply-chain flow programs, open ERP schemas, traceability standards, and controlled schema corruptions.
5. Downstream demonstrations for inventory reconstruction, waste detection, carbon activity-data accounting, and disruption analysis.

The idea is potentially suitable for ICLR only if it is presented as a general learning problem with theory, out-of-distribution evaluation, and hard solver baselines. If reduced to an LLM that renames ERP columns, the contribution would be incremental and the venue fit would be weak.

Decision Requested from the Supervisor

Approve a focused validation stage around the core claim: can invariant-grounded program induction recover semantics under severe schema shift, and can it outperform or scale beyond classical constraint solvers without sacrificing formal ambiguity detection? Carbon and resilience should remain downstream applications, not separate modules competing for the paper's novelty budget.

1. Problem and Motivation

1.1 The hidden prerequisite behind supply-chain AI

Forecasting, optimization, carbon estimation, and resilience analysis assume that each observation already has a stable meaning. Real cross-enterprise data violates this assumption. Procurement, warehouse, manufacturing, transport, returns, and finance systems encode the same physical process using incompatible schemas. NIST identifies semantic interoperability and low-effort integration of traceability data as a central need for sustainable and efficient value chains [9].

Conventional data integration relies on labels, metadata, manually written mappings, or similarity between column values. These signals are useful but fragile. Headers can be abbreviated or misleading; numeric columns with different roles can have similar distributions; and pairwise matching cannot ensure that the resulting set of mappings describes a physically possible process.

1.2 The invariant opportunity

Supply-chain representations are heterogeneous, but the system they describe is constrained. Inventory cannot increase without an inbound, production, return, or correction event. Finished goods require input materials. Shipments leave a node before they arrive downstream, subject to a nonnegative lag. Scrap and returns affect both mass balance and environmental activity data. These relations create a structured supervisory signal that remains meaningful after headers are erased.

$$I_{t+1} = I_t + \text{receipts}_t + \text{production}_t + \text{returns}_t - \text{shipments}_t - \text{consumption}_t - \text{scrap}_t + \text{eps}_t$$

The residual eps is not assumed to be zero: enterprise ledgers contain missing events, timing differences, cycle-count adjustments, and data-entry errors. The research challenge is therefore not equation fitting. It is robust discovery of the semantics and program that make many coupled equations simultaneously plausible.

1.3 Research question

CENTRAL QUESTION

If table and column names are removed, can an AI system reconstruct the semantics of supply-chain data from the conservation laws and temporal dependencies that the physical process cannot violate?

1.4 Scope boundary

- **In scope:** semantic roles, units and scales, sign conventions, joins and keys, time lags, event types, and executable stock-flow programs.
- **In scope:** formal recovery conditions, equivalence classes, and ambiguity certificates.
- **Demonstrations:** activity-based carbon estimation, waste and leakage detection, inventory reconstruction, and disruption propagation.
- **Out of scope for the first paper:** a complete enterprise data platform, autonomous procurement, or a universal carbon database.

2. Formal Problem Formulation

2.1 Observed data and latent semantics

Let $D = \{D^{(1)}, ..., D^{(M)}\}$ be a collection of relational tables from one or more organizations. Headers and business descriptions may be unavailable or treated only as optional priors. Each observed numeric column $x^{(m)}_{j,t}$ is generated from a latent physical variable $z_{k,t}$ through an unknown semantic assignment, scale, sign, and lag:

$$x^{(m)}_{j,t} = a_j \, z_{\mathrm{map}(j),\, t - \tau(j)} + \mathrm{noise}_{j,t}$$

The unknown program P additionally specifies primary and foreign keys, aggregation rules, node-product relationships, and a set of conservation equations. The task is to infer P and a calibrated set of alternative programs P_{eq} that are observationally indistinguishable.

2.2 Output object

Recovered object	What it contains	Why it matters
Semantic mapping	Role of each table and column: stock, receipt, shipment, production, consumption, return, scrap, identifier, time.	Transforms raw data into a canonical vocabulary.
Measurement mapping	Unit, scale, sign, aggregation window, and conversion factor.	Prevents physically inconsistent totals and carbon estimates.
Relational mapping	Keys, joins, entity correspondences, and event lineage.	Reconstructs the multi-table process rather than isolated columns.
Temporal mapping	Posting delay, transport lag, production lead time, and permissible ordering.	Separates delayed records from impossible flows.
Flow program	Executable equations linking inventory, material transformations, transport, returns, and waste.	Enables verification and downstream simulation.
Ambiguity certificate	All materially different programs within an evidence tolerance, or a proof of unique recovery under stated assumptions.	Avoids false certainty when data do not identify the semantics.

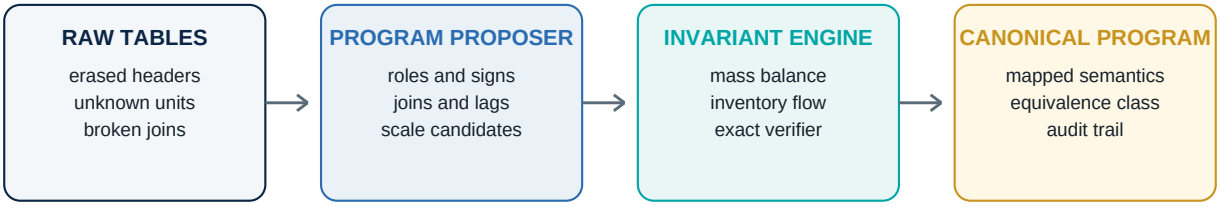
2.3 Learning objective

A candidate program is scored by robust invariant residual, semantic plausibility, relational consistency, temporal validity, and program complexity. One representative objective is:

$$L(P) = \lambda_{\mathrm{inv}} L_{\mathrm{inv}} + \lambda_{\mathrm{join}} L_{\mathrm{join}} + \lambda_{\mathrm{time}} L_{\mathrm{time}} + \lambda_{\mathrm{prior}} L_{\mathrm{prior}} + \beta\, \mathrm{Complexity}(P)$$

The metadata prior may use headers or descriptions when available, but the main benchmark will include a strict header-erased condition. A robust loss and explicit slack variables handle missing or delayed postings.

3. Proposed Method



Learned proposals make search scalable; invariants and the verifier make outputs defensible.

Figure 1. Proposed hybrid architecture for INVARIANT-SC.

3.1 Stage A - schema-agnostic evidence encoder

For each column and table, the encoder summarizes value distributions, sparsity, temporal behavior, cardinality, monotonicity, cross-correlation, candidate units, and optional text metadata. A relational encoder propagates evidence across candidate key relationships. The goal is not to decide the mapping independently for each column, but to propose a globally coherent set of roles.

3.2 Stage B - structured program proposal

A learned proposal network predicts a small set of candidate assignments for roles, joins, signs, scales, and lags. Discrete assignments can be relaxed during training with a doubly stochastic mapping, while final decoding remains discrete. Synthetic schema transformations provide supervision without requiring large collections of manually mapped proprietary ERP systems.

3.3 Stage C - differentiable invariant interpreter

Each candidate is executed as a stock-flow program. The interpreter performs temporal joins, aggregates events, converts units, and evaluates material-balance, inventory-balance, and transformation constraints. Residuals are computed over multiple nodes, products, and time windows so that a locally plausible mapping cannot survive if it breaks the global process.

3.4 Stage D - exact verification and ambiguity

A CP-SAT or mixed-integer verifier searches the neighborhood of the learned proposals. It returns either a unique optimum under the stated tolerance or an equivalence class of alternatives. This hybrid design has two purposes: the learned model narrows an otherwise combinatorial search, while the verifier produces a defensible certificate instead of an opaque confidence score.

NON-NEGOTIABLE DESIGN PRINCIPLE

When the observed process is symmetric, the correct output is not a confident guess. It is an explicit statement that two or more semantic programs are observationally equivalent, together with the additional measurement needed to separate them.

3.5 Canonical output and audit trail

The final output is a versioned canonical flow graph with evidence for every mapping: source columns, unit conversions, inferred joins, lag estimates, invariant residuals, and unresolved alternatives. This object can be exported toward traceability models such as EPCIS rather than replacing them [10].

4. Theory and Identifiability Agenda

The theoretical contribution must be stated as a target until proofs are complete. The paper should not claim that every real ERP schema is recoverable. Instead, it should characterize exactly when recovery is possible and what ambiguity remains when those conditions fail.

4.1 Target Theorem A - unique recovery

PROPOSED STATEMENT

For a connected latent flow program with sufficiently exciting observations, identifiable unit-ratio candidates, correctly bounded lags, and no nontrivial program automorphism, the semantic assignment, signs, relative scales, joins, and lags are uniquely recoverable up to an explicitly defined global equivalence.

4.2 Target Theorem B - impossibility under symmetry

If two variables or subgraphs are related by an automorphism that preserves every observed invariant and distribution, no algorithm can distinguish their semantic labels from the available data. The verifier should return the corresponding equivalence class rather than one arbitrary labeling.

4.3 Target Theorem C - robust recovery

Under bounded missingness, noise, and posting delay, the error of the recovered continuous parameters should be bounded by the condition number of the excitation matrix and the margin between the best and second-best programs. This connects data diversity to semantic recoverability.

4.4 Assumptions to test, not hide

Assumption	Operational meaning	Failure mode / response
Excitation	Flows vary enough over time to distinguish roles.	Constant or perfectly correlated series create ambiguity; return an equivalence class.
Connectivity	Candidate joins connect the observed parts of the process.	Disconnected components can only be identified locally.
Bounded delay	Posting and transport lags fall inside a finite search range.	Use broader windows or mark the event relation unresolved.
Unit family	Plausible scale ratios belong to a finite or learnable set.	Mixed units within one column require segmentation or are rejected.
Sparse corruption	Most events are valid; missingness and adjustments are not adversarially dominant.	Escalate residual and abstain rather than force a program.

A theorem-plus-certificate package is the key difference between INVARIANT-SC and ordinary schema matching. It turns uncertainty from a weakness into a formally specified output.

5. Novelty and Closest Prior Work

The novelty claim is **not** that schema matching, conservation-aware learning, or supply-chain modeling is individually new. The proposed contribution is their joint formulation as identifiable latent flow-program induction under schema shift. The current literature screen finds strong neighboring work, but no exact match to the complete problem below. This is a provisional assessment and should be followed by a formal systematic search across ICLR, ICML, NeurIPS, SIGMOD, VLDB, ICDE, KDD, AAAI, and operations journals.

Closest line of work	What it already does	Boundary retained by INVARIANT-SC
Matchmaker, ICML 2025 [1]	Zero-shot language-model program for schema matching.	Does not jointly infer physical roles, units, joins, lags, executable flow laws, and formal non-identifiability.
SATO and Doduo [3, 4]	Predict semantic column types and relations from table content.	Uses learned labels and table context; does not discover a coupled dynamic supply-chain program from conservation laws.
OmniMatch, PVLDB 2025 [5]	Discovers joinable column pairs using multiple similarities and a GNN.	Joinability is one component; the proposed task requires global semantics, temporal alignment, units, signs, and program validity.
Relational Deep Learning, ICML 2024 [2]	Learns on connected relational tables and reduces manual feature engineering.	Generally starts from known table entities and primary-foreign-key structure; INVARIANT-SC must first infer those semantics and links.
ProbConserv, ICML 2023 [6]	Enforces a known conservation law in scientific machine learning.	The law is used to discover an unknown data interpretation and program, not only to improve prediction under a known physical model.
Supply-chain production functions, AAAI 2025 [7]	Infers hidden input-output production relations from known firm-product transaction graphs.	Assumes meaningful nodes, products, and transactions; INVARIANT-SC targets anonymized heterogeneous relational observations and mapping ambiguity.
NIST and GS1 traceability models [9, 10]	Define interoperable target semantics and event models.	Standards require a mapping from legacy data; the proposed model infers and verifies that mapping.

DEFENSIBLE NOVELTY STATEMENT

To the best of the present literature screen, no existing method jointly recovers semantic roles, units, signs, joins, temporal lags, and an executable supply-chain flow program from anonymized relational tables while also returning an identifiability or ambiguity certificate.

5.1 What would destroy the novelty

- Framing the work as an ERP column-name mapper or generic LLM agent.
- Evaluating only on randomly renamed columns while keeping joins, units, and program topology fixed.
- Omitting classical optimization and constraint-discovery baselines.
- Claiming uniqueness without a theorem, equivalence definition, or verifier.
- Adding unrelated carbon and resilience modules without showing that they depend on the recovered canonical program.

6. Benchmark and Data Plan

6.1 Benchmark principle

A convincing benchmark must test semantic recovery under transformations that occur in enterprise data, not merely paraphrased headers. Each example begins with a known canonical flow program. A corruption engine then generates heterogeneous schemas while preserving the underlying physical process. Ground truth remains available for exact program-level evaluation.

6.2 Proposed data sources

Source	Role in the benchmark	Availability
SupplySim [12]	Generate temporal production and transaction graphs with controlled topology and disruptions.	Open-source code and simulator.
ERPNext and Odoo [13, 14]	Provide realistic open ERP entities, workflows, posting patterns, and schema vocabulary for scenario design.	Open-source repositories.
GS1 EPCIS examples [10]	Define canonical visibility-event targets, including object, aggregation, transaction, transformation, and association events.	Open standard and example artefacts.
EPA USEEIO [16]	Attach reproducible economic-sector emission factors to reconstructed activity data.	Open model and code.
Synthetic ledger generator	Create exact ground truth for units, signs, joins, lags, missing events, and ambiguity classes.	To be released with the benchmark.

6.3 Corruption and shift suite

Axis	Difficulty levels
Labels	Canonical names; aliases; abbreviations; random identifiers; fully erased headers.
Structure	One wide table; normalized tables; denormalized snapshots; event tables; split ledgers; extra decoy tables.
Measurement	Unit changes from 10^-3 to 10^3; sign flips; rounding; currency conversion; aggregation windows; mixed-unit failure cases.
Time	Unknown lags; delayed postings; irregular sampling; clock offsets; out-of-order events.
Quality	5-30% missing events; duplicates; corrections; sparse corruptions; unrelated decoy series.
Process	Returns, scrap, substitution, rework, multi-output production, transport consolidation, and inventory adjustment.
Generalization	Unseen schema; unseen organization; unseen topology; unseen generating program; unseen industry mix.

6.4 Required data splits

1. **Schema OOD:** new table layouts and names generated from known program families.
2. **Program OOD:** new flow equations, returns or transformation structures absent from training.
3. **Topology OOD:** larger and structurally different multi-node networks.
4. **Source OOD:** schemas patterned after an unseen ERP family.
5. **Ambiguity split:** deliberately symmetric cases where the correct answer is an equivalence class.

7. Experimental Design

7.1 Baselines

The benchmark must include both machine-learning and non-learning competitors:

- Header and value similarity with Hungarian assignment.
- Semantic column typing using SATO or Doduo-style encoders [3, 4].
- Zero-shot LLM schema matching, including Matchmaker-style programs [1].
- Join discovery using OmniMatch or comparable similarity ensembles [5].
- Dependency discovery plus unit and lag heuristics.
- Pure CP-SAT or mixed-integer program search over roles, signs, scales, joins, and lags.
- LLM plus exact solver, without the proposed learned invariant architecture.
- Oracle variants with known joins, known units, or known program family to measure where difficulty enters.

7.2 Metrics

Metric	Definition / purpose
Role macro-F1	Per-role semantic recovery, robust to class imbalance.
Exact program recovery	All roles, signs, units, joins, and lags correct for an instance.
Join edge F1	Accuracy of the inferred relational graph.
Scale error	Absolute log-ratio error of inferred conversion factors.
Lag error	Mean absolute error in posting or transport lag.
Invariant residual	Normalized mass and inventory imbalance after reconstruction.
Selective risk	Error versus coverage when the system abstains on ambiguous instances.
Ambiguity-set quality	Does the returned equivalence class contain exactly the observationally valid programs?
Runtime and scaling	Columns, tables, candidate joins, nodes, and time windows handled per unit compute.
Downstream error/regret	Inventory, emissions, waste, and disruption estimates relative to canonical ground truth.

7.3 Main experiments

1. **Header-erased recovery:** measure how much signal remains when language is removed.
2. **Schema-shift scaling:** increase tables and columns from 50 to 500 while varying join density.
3. **Noise and missingness:** sweep missing events from 5% to 30%, add corrections and posting delay.
4. **Program OOD:** test on new production, return, and waste structures.
5. **Certificate validity:** create known symmetries and verify that the method abstains with the correct equivalence class.
6. **Downstream transfer:** train the same forecasting or carbon estimator on raw, conventionally matched, and INVARIANT-SC canonicalized data.

7.4 Ablations

- Remove text metadata; remove values; remove temporal structure; remove conservation loss.
- Replace exact verification with confidence scoring.
- Remove learned proposal and run solver-only search.
- Use local pairwise constraints instead of the global program.
- Train without program-OOD corruptions or ambiguity cases.

8. Sustainability and Resilience Applications

The project is an umbrella because one recovered physical program supports several analyses. These are not separate novelty claims; they are tests of whether the semantic layer is useful.

8.1 Carbon activity-data reconstruction

GHG Protocol calculations commonly multiply activity data by emission factors [11]. The difficult part is often obtaining trustworthy activity data with consistent units, categories, boundaries, and lineage. INVARIANT-SC first reconstructs material, energy, transport, purchased-goods, sold-product, and waste activity. Emission factors are then attached with their source and boundary. The model should never invent an emission factor or treat estimated carbon as a conserved physical quantity.

$$\text{Emissions} = \sum_a \text{Activity}_a \times \text{EmissionFactor}_a$$

8.2 Waste, leakage, and circularity

Persistent unexplained flow residual can indicate scrap, shrinkage, data loss, or an incorrect mapping. Returns, rework, recycling, and by-products can be represented explicitly, allowing circular flows to be measured instead of treated as noise.

8.3 Inventory visibility and resilience

A canonical stock-flow graph provides the state variables needed to quantify disruption propagation, time to recovery, service loss, and inventory exposure. Recent work predicts resilience from known inventory trajectories and hypergraph topology [8]; INVARIANT-SC addresses the upstream question of recovering those consistent trajectories and relationships from heterogeneous records.

8.4 Multi-tier traceability

The inferred event lineage can be mapped to EPCIS event types and used to identify which records require human validation. This complements standards adoption by reducing the manual cost of bringing legacy data into a shared semantic model [9, 10].

9. Preliminary Feasibility Signal

A small internal enumerative smoke test was run to determine whether inventory invariants contain enough signal to recover erased roles. It is not a learned model, not a benchmark result, and not evidence of publication novelty. It only tests whether the basic inverse problem is plausible.

Setting	Value / result
Synthetic ledgers	200
Corruptions	Random headers and permutations; scales from 10^-3 to 10^3; 8% missing cells; 1.5% measurement noise; unrelated decoy time series.

Setting	Value / result
Begin/end inventory roles	100% recovered
Correct flow-column set	99.5% recovered
Shipment/outflow role	100% recovered
Joint structural recovery	99.5%
Median best-to-second interpretation margin	31.8x

INTERPRETATION

The invariant signal exists in a toy setting. The next experiment must show that a learned proposal plus verifier scales to large, multi-table, noisy, program-OOD settings and improves over a short classical solver.

10. Risks, Failure Modes, and Kill Tests

A strong proposal should specify what evidence would falsify the idea. The following tests prevent the team from spending months polishing an integration demo that does not contain a research contribution.

Risk	Why it matters	Mitigation / kill criterion
Classical solver wins	The learned architecture may be unnecessary.	Benchmark a carefully tuned CP-SAT/MILP solver. If it matches accuracy and scaling across the full benchmark, pivot to a theory/benchmark paper or stop the method claim.
Insufficient identifiability	Many programs may fit normal ledger data.	Quantify equivalence classes; add interventions or richer observations. Kill unique-recovery claims where symmetry remains.
Benchmark shortcut	Models may exploit generator artefacts or hidden metadata.	Use multiple generators, ERP-inspired schemas, header-erased tests, decoy variables, and source-OOD evaluation.
Poor program OOD	A model that memorizes templates is not a general semantic learner.	Hold out generating programs and topology families; require meaningful recovery above non-learning baselines.
Carbon overclaim	Emission factors and boundaries are not inferred by mass balance.	Recover auditable activity data; report factor source, boundary, and uncertainty separately.
Database-paper perception	ICLR reviewers may see schema engineering rather than ML.	Lead with latent representation discovery, identifiability, structured induction, and cross-schema generalization.
Enterprise realism gap	Synthetic ledgers can be cleaner than real posting systems.	Include delayed postings, corrections, mixed aggregation, and open ERP workflow cases; seek one external blind schema if possible.

10.1 Mandatory fatal test

PROCEED ONLY IF THIS SURVIVES

Use 50-500 columns across normalized and denormalized tables with unknown keys, joins, and lags; include returns, scrap, substitutions, delayed postings, 5-30% missing events, mixed-unit failure cases, decoys, unseen ERP schemas, and unseen program topologies. Compare against solver-only, LLM, schema-matching, semantic-typing, and join-discovery baselines.

10.2 Success gates

- Exact program recovery materially exceeds every component baseline on schema OOD and program OOD.
- The learned proposal reduces exact-search cost substantially without reducing certificate correctness.
- Ambiguous cases are detected with strong selective-risk and set-coverage performance.
- Recovered programs reduce downstream inventory, waste, or carbon error relative to raw and conventionally matched data.
- At least one theorem or impossibility result is complete and nontrivial.

11. Expected Contributions

Contribution	Concrete paper artefact
Task	Formal definition of physics-grounded semantic schema induction and its ambiguity-aware output.
Method	Learned proposal network, differentiable flow interpreter, robust temporal alignment, and exact verifier.
Theory	Recovery conditions, impossibility under automorphism, and robustness/margin analysis.
Benchmark	Open generator, ERP-inspired schemas, corruption suite, OOD splits, and exact ground truth.
Evidence	Comparison with LLM, semantic-typing, join-discovery, heuristic, and optimization baselines.
Impact	Auditable downstream studies for activity-based emissions, waste, inventory, and resilience.

12. Research Work Packages

1. **Formal specification:** define the program language, equivalence relation, observation model, and target theorems.
2. **Prior-art closure:** complete a systematic claim matrix across ML, databases, process mining, system identification, and operations research.
3. **Benchmark generator:** implement canonical programs, schema transformations, ambiguity cases, and OOD splits.
4. **Baseline suite:** build the smallest strong classical and ML baselines before the proposed architecture.
5. **Hybrid model:** add learned proposals only where they improve scaling or OOD generalization over exact search.
6. **Theory and certificates:** prove recovery/impossibility results and validate certificate correctness empirically.
7. **Downstream validation:** quantify the value of canonicalization for emissions activity data, waste, inventory, and disruption analysis.

13. Venue Positioning

For ICLR, the paper should be positioned as a general problem in structured representation learning: how to recover latent semantics and programs from relational observations using weak physical supervision. The supply-chain domain supplies a high-impact, mathematically structured test bed. The paper should emphasize identifiability, program OOD, abstention under non-identifiability, and the hybrid learned-symbolic method.

If theory or OOD generalization does not materialize, the surviving benchmark or deployment may point to a database, data-mining, or industry venue. Venue choice should follow the contribution, not the project name.

14. Recommended Research Position

RECOMMENDATION

Proceed with INVARIANT-SC as the flagship direction, conditional on the fatal test. Protect the core novelty: joint latent flow-program recovery plus identifiability. Treat carbon, waste, inventory, traceability, and resilience as one coherent evidence chain built on the recovered program.

This formulation has genuine research depth because it asks a new inverse problem, admits theory, supports a hard benchmark, and addresses an operational barrier recognized by standards and supply-chain research. It is not guaranteed to be accepted, and absolute novelty cannot be certified before exhaustive prior-art review. However, the present formulation is sufficiently differentiated and falsifiable to justify serious research effort.

References

[1] N. Seedat and M. van der Schaar. Bootstrapping Self-Improvement of Language Model Programs for Zero-Shot Schema Matching. ICML, 2025. PMLR page.

[2] M. Fey et al. Position: Relational Deep Learning - Graph Representation Learning on Relational Databases. ICML, 2024. PMLR page.

[3] Y. Suhara et al. Annotating Columns with Pre-trained Language Models. 2021. arXiv.

[4] D. Zhang et al. SATO: Contextual Semantic Type Detection in Tables. 2019. arXiv.

[5] C. Koutras et al. OmniMatch: Joinability Discovery in Data Products. PVLDB 18(11), 2025. PVLDB PDF.

[6] D. Hansen et al. Learning Physical Models that Can Respect Conservation Laws. ICML, 2023. PMLR page.

[7] S. Chang et al. Learning Production Functions for Supply Chains with Graph Neural Networks. AAAI, 2025. AAAI page.

[8] Z. Shen et al. Resilience Inference for Supply Chains with Hypergraph Neural Network. AAAI, 2026. AAAI page.

[9] National Institute of Standards and Technology. Enabling and Using Traceability and Data Linking for Sustainable and Efficient Supply Chains. Updated 2026. NIST project.

[10] GS1. EPCIS Standard, version 2.0.1. 2025. Official standard.

[11] GHG Protocol. Scope 3 Calculation Guidance and quantification guidance. Official guidance.

[12] Stanford SNAP. SupplySim and supply-chain production-function code. GitHub repository.

[13] Frappe Technologies. ERPNext. GitHub repository.

[14] Odoo S.A. Odoo. GitHub repository.

[15] Amazon Science et al. OmniMatch implementation. GitHub repository.

[16] U.S. Environmental Protection Agency. USEEIO: U.S. Environmentally-Extended Input-Output Models. GitHub repository.

Note on evidence: references were selected from official conference proceedings, standards bodies, government sources, and project repositories. The novelty conclusion remains provisional until the planned systematic review is complete.